

MATERIAL EXTRA

Tipos Genéricos e Utilitários

TIPOS GENÉRICOS

Embora os genéricos sejam compatíveis com muitas linguagens de programação (incluindo o TypeScript), eles não estão disponíveis no momento no JavaScript. Portanto, para alguns codificadores de JavaScript, os genéricos são um mistério. Agora que você está no caminho para ser um codificador TypeScript, vamos revelar os mistérios dos genéricos e adicionar essas ferramentas úteis ao seu kit de ferramentas de programação.

Em aulas anteriores, no roteiro de aprendizagem TypeScript, você aprendeu a aplicar anotações de tipo às suas interfaces, funções e classes a fim de criar componentes fortemente tipados. Mas e se você quiser criar um componente que possa trabalhar em vários tipos em vez de em apenas um? Você poderia usar o tipo **any**, mas assim você perderia o poder por trás do sistema de verificação de tipo do TypeScript.

Os genéricos são modelos de código que você pode definir e reutilizar em toda a base de código. Eles fornecem uma forma de informar a funções, classes ou interfaces que tipo você deseja usar ao chamá-las. Você pode pensar nisso como sendo da mesma maneira que os argumentos são passados para uma função, exceto pelo fato de que um genérico permite que você informe ao componente qual tipo ele deve esperar no momento que é chamado.

Crie funções genéricas quando seu código for uma função ou classe que:

- Funciona com uma variedade de tipos de dados.
- Usa esse tipo de dados em vários lugares.

Os genéricos podem:

- Fornecer mais flexibilidade ao trabalhar com tipos.
- Permitir a reutilização de código.
- Reduzir a necessidade de usar o tipo **any**.

Por que usar os genéricos?

Para entender por que você poderia usar os genéricos, é útil ver um exemplo.

A função **getArray** gera uma matriz de itens do tipo **any**.

```
function getArray(items: any[]): any[] {  
  return new Array().concat(items);  
}
```

Em seguida, a variável **numberArray** é declarada chamando a função **getArray** e passando a ela uma matriz de números, enquanto a variável **stringArray** é declarada com uma matriz de cadeias de caracteres. No entanto, como o tipo **any** é usado, não há nada que impeça o código de enviar um **string** por push para o **numberArray** ou um **number** para o **stringArray**.

```
let numberArray = getArray([5, 10, 15, 20]);
let stringArray = getArray(['Cats', 'Dogs', 'Birds']);
numberArray.push(25); // OK
stringArray.push('Rabbits'); // OK
numberArray.push('This is not a number'); // OK
stringArray.push(30); // OK
console.log(numberArray); // [5, 10, 15, 20,
25, "This is not a number"]
console.log(stringArray); // ["Cats", "Dogs",
"Birds", "Rabbits", 30]
```

E se você quiser determinar o tipo dos valores que a matriz vai conter quando chamar a função e fazer com que o TypeScript verifique o tipo dos valores que você passar para ele a fim de garantir que eles sejam desse tipo? É nesse momento que os genéricos entram em cena.

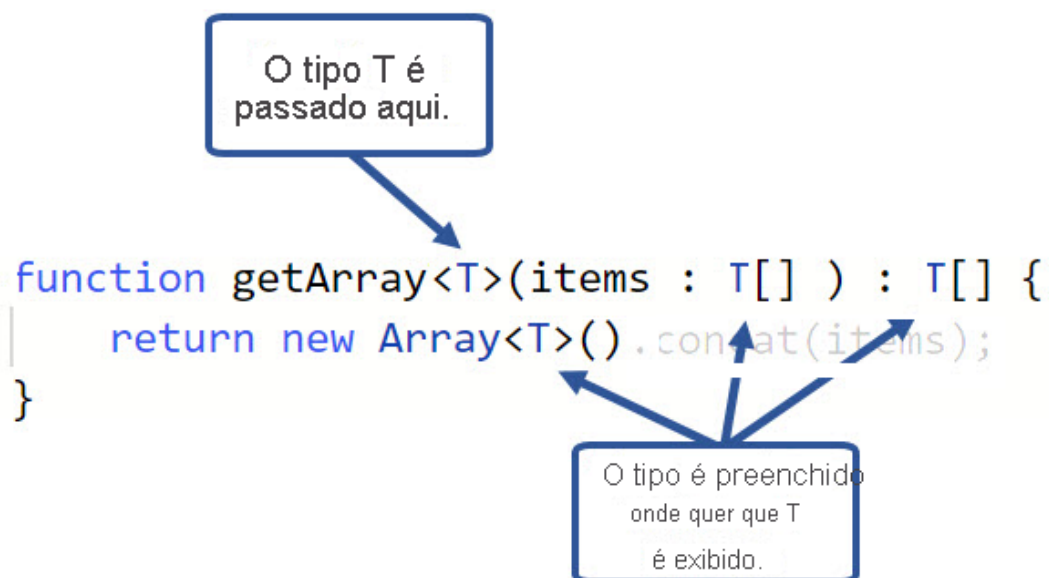
Este exemplo reescreve a função **getArray** usando genéricos. Agora ele pode aceitar qualquer tipo que você especificar ao chamar a função.

```
function getArray<T>(items: T[]): T[] {
  return new Array<T>().concat(items);
}
```

Os genéricos definem uma ou mais variáveis de tipo para identificar o tipo ou tipos que serão passados para o componente, colocados entre colchetes angulares (< >). Você também verá variáveis de tipo mencionadas como parâmetros de tipo ou parâmetros genéricos.

No exemplo acima, a variável de tipo na função é chamada **<T>**. **T** é um nome comumente usado para um genérico, mas você pode nomeá-lo como desejar.

Depois de especificar a variável de tipo, você pode usá-la no lugar do tipo em parâmetros, no tipo de retorno ou em qualquer outro lugar na função em que você adicionaria uma anotação de tipo.



A variável de tipo **T** poderá ser usada sempre que a anotação de tipo for necessária. Na função **getArray**, ela é usada para especificar o tipo para o parâmetro **items**, o tipo de retorno da função e para retornar uma nova matriz de itens.

Para chamar a função e passar um tipo para ela, acrescente **<type>** ao nome da função. Por exemplo, **getArray<number>** instrui a função a aceitar apenas uma matriz de valores **number** e retornar uma matriz de valores **number**. Como o tipo foi especificado como um **number**, o TypeScript esperará que valores **number** sejam passados para a função e eles vão gerar um erro se forem algo diferente.

OBSERVAÇÃO: Se você omitir a variável de tipo ao chamar a função, o TypeScript vai inferir o tipo. No entanto, isso só funciona se os dados são simples. A passagem de matrizes ou objetos infere o tipo `any` e elimina as verificações de tipo.

Neste exemplo, com as declarações de variável para **numberArray** e **stringArray** atualizadas para chamar a função com o tipo desejado, o TypeScript impede que os itens inválidos sejam adicionados à matriz.

```
let numberArray = getArray<number>([5, 10, 15, 20]);
numberArray.push(25); // OK
numberArray.push('This is not a number'); // Generates a compile
time type check error

let stringArray = getArray<string>(['Cats', 'Dogs', 'Birds']);
stringArray.push('Rabbits'); // OK
stringArray.push(30); // Generates a compile time type
check error
```

Como usar variáveis de vários tipos

Você não está limitado a usar apenas uma variável de tipo em seus componentes genéricos.

Por exemplo, a função **identity** aceita dois parâmetros, **value** e **message**, e retorna o parâmetro **value**. Você pode usar dois genéricos **T** e **U**, para atribuir tipos diferentes a cada parâmetro e para o tipo de retorno.

A variável **returnNumber** é inicializada chamando a função **identity** com **<number, string>** como os tipos para os argumentos **value** e **message**, **returnString** é inicializada chamando-a com **<string, string>** e **returnBoolean** é inicializada chamando-a com **<boolean, string>**. Ao usar essas variáveis, o TypeScript poderá fazer a verificação de tipo dos valores e, se houver um conflito, retornará um erro em tempo de compilação.

```
function identity<T, U> (value: T, message: U) : T {  
    console.log(message);  
    return value  
}  
  
let returnNumber = identity<number, string>(100, 'Hello!');  
let returnString = identity<string, string>('100', 'Hola!');  
let returnBoolean = identity<boolean, string>(true, 'Bonjour!');  
  
returnNumber = returnNumber * 100;    // OK  
returnString = returnString * 100;    // Error: Type 'number' not  
assignable to type 'string'  
returnBoolean = returnBoolean * 100; // Error: Type 'number' not  
assignable to type 'boolean'
```

Usar os métodos e as propriedades de um tipo genérico

Ao usar variáveis de tipo para criar componentes genéricos, você pode usar apenas as propriedades e métodos de objetos que estão disponíveis para todos os tipos. Isso impede a ocorrência de erros quando você tenta executar uma operação em um valor de parâmetro que é incompatível com o tipo que está sendo passado para ele.

Se você adicionar a instrução `let result: T = value + value` à função `identity`, o TypeScript vai gerar o erro **O lado esquerdo de uma operação aritmética precisa ser do tipo 'any', 'number', 'bigint' ou um tipo enumerado** porque não sabe qual valor será passado para ele em runtime. Se você passasse um valor não numérico, a expressão geraria um erro, portanto, o TypeScript informa você sobre o problema em tempo de compilação.

```
function identity<T, U>(value: T, message: U): T {  
  let result: T = value + value; // Error  
  console.log(message);  
  return result;  
}
```

Como usar restrições genéricas para limitar tipos

A função `identity` aceitará qualquer tipo que você escolher para passar para as variáveis de tipo. Mas, nesse caso, você deve restringir os tipos que o parâmetro `value` pode aceitar para um intervalo de tipos em que você pode executar uma operação de adição, em vez de aceitar qualquer tipo possível. Isso é chamado de **restrição genérica**.

Há várias maneiras de fazer isso, dependendo da variável de tipo. Uma delas é declarar um **type** personalizado como uma tupla e **extend** a variável de tipo com o tipo personalizado. O exemplo a seguir declara **ValidTypes** como uma tupla com um **string** e um **number**. Em seguida, ele estende **T** com o novo tipo. Agora, você só pode passar os tipos **number** ou **string** para a variável de tipo.

```
type ValidTypes = string | number;

function identity<T extends ValidTypes, U>(value: T, message: U): T {
  let result: T = value + value; // Error
  console.log(message);
  return result;
}

let returnNumber = identity<number, string>(100, "Hello!"); // OK
let returnString = identity<string, string>("100", "Hola!"); // OK

// Error: Type 'boolean' does not satisfy the constraint 'ValidTypes'.
let returnBoolean = identity<boolean, string>(true, "Bonjour!");
```

Você também pode restringir um tipo à propriedade de outro objeto. Este exemplo usa **extends** com o operador **keyof**, que usa um tipo de objeto e produz uma união de cadeia de caracteres ou de literal numérico das chaves dele. Aqui, **K extends keyof T**, garantindo que o parâmetro de chave seja do tipo correto para o tipo atribuído a **pet**.

```
function getPets<T, K extends keyof T>(pet: T, key: K) {
  return pet[key];
}

let pets1 = { cats: 4, dogs: 3, parrots: 1, fish: 6 };
let pets2 = { 1: "cats", 2: "dogs", 3: "parrots", 4: "fish" };

console.log(getPets(pets1, "fish")); // Returns 6
console.log(getPets(pets2, "3")); // Error
```

Usar proteções de tipo com genéricos

Você observará que o TypeScript ainda gera um problema com a expressão `value + value` na função `identity`. Mas agora você sabe que apenas os tipos `number` e `string` podem ser passados para a função.

Você pode usar a proteção de tipo `typeof` em um bloco `if` para verificar o tipo do parâmetro `value` antes de executar uma operação, conforme mostrado no exemplo a seguir. O TypeScript pode determinar, com base na instrução `if`, se a operação funcionará com os valores fornecidos no bloco.

```
type ValidTypes = string | number;
function identity<T extends ValidTypes, U> (value: T, message: U) {
    let result: ValidTypes = '';
    let typeValue: string = typeof value;

    if (typeof value === 'number') {           // Is it a number?
        result = value + value;                // OK
    } else if (typeof value === 'string') {    // Is it a string?
        result = value + value;                // OK
    }

    console.log(`The message is ${message} and the function returns a
    ${typeValue} value of ${result}`);

    return result
}

let numberValue = identity<number, string>(100, 'Hello');
let stringValue = identity<string, string>('100', 'Hello');

console.log(numberValue);                     // Returns 200
console.log(stringValue);                     // Returns 100100
```

OBSERVAÇÃO: Você só pode usar uma proteção de tipo **typeof** para verificar os tipos primitivos **string**, **number**, **bigint**, **function**, **boolean**, **symbol**, **object** e **indefinido**. Para verificar o tipo de uma classe, use uma proteção de tipo **instanceof**.

TIPOS UTILITÁRIOS

Os Tipos Utilitários são ferramentas poderosas fornecidas pelo TypeScript para manipular e transformar tipos existentes. Eles ajudam a criar novos tipos com base em tipos já definidos, reduzindo a repetição de código e tornando o desenvolvimento mais eficiente. Vejamos alguns dos principais tipos utilitários disponíveis no Typescript.

Partial<T>: Transforma todas as propriedades de um tipo em opcionais.

💡 **Quando usar:** Quando você não precisa definir todas as propriedades de um objeto.

```
interface User {
  id: number;
  name: string;
  email: string;
}

const updateUser = (user: Partial<User>) => {
  console.log(user);
};

updateUser({ name: "John" }); // Tudo bem, apenas "name" foi passado
```

Required<T>: Transforma todas as propriedades de um tipo em obrigatórias.

💡 **Quando usar:** Quando você quer garantir que todas as propriedades sejam preenchidas.

```
interface User {
  id?: number;
  name?: string;
  email?: string;
}

const createUser = (user: Required<User>) => {
  console.log(user);
};

createUser({ name: "John" }); // Erro: faltam as propriedades "id" e "email"
```

Readonly<T>: Transforma todas as propriedades de um tipo em somente leitura, impedindo alterações.

💡 **Quando usar:** Quando deseja proteger um objeto de ser modificado.

```
interface User {
  id: number;
  name: string;
}

const user: Readonly<User> = { id: 1, name: "John" };

user.id = 2; // Erro: a propriedade "id" é somente leitura
```

Pick<T, K>: Cria um novo tipo baseado em um tipo existente, selecionando apenas algumas propriedades.

💡 **Quando usar:** Quando você precisa trabalhar apenas com partes de um tipo.

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
}  
  
type UserPreview = Pick<User, "id" | "name">;  
const user: UserPreview = { id: 1, name: "John" }; // "email" não é  
necessário
```

Omit<T, K>: Cria um novo tipo baseado em um tipo existente, removendo algumas propriedades.

💡 **Quando usar:** Quando você quer excluir propriedades específicas de um tipo.

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
}  
  
type UserWithoutEmail = Omit<User, "email">;  
const user: UserWithoutEmail = { id: 1, name: "John" }; // "email" foi  
removido
```

Record<K, T>: Cria um tipo de objeto em que todas as chaves têm o mesmo tipo.

💡 **Quando usar:** Quando você precisa de um objeto com chaves dinâmicas, mas todas as chaves têm o mesmo valor.

```
type Role = "admin" | "user" | "guest";
type Permissions = Record<Role, boolean>;

const userPermissions: Permissions = {
  admin: true,
  user: false,
  guest: false,
};
```

ReturnType<T>: Obtém o tipo de retorno de uma função.

💡 **Quando usar:** Quando você quer reutilizar o tipo de retorno de uma função.

```
function getUser() {
  return { id: 1, name: "John" };
}

// { id: number, name: string }
type UserReturnType = ReturnType<typeof getUser>;
```

Parameters<T>: Obtém uma tupla com os tipos dos parâmetros de uma função.

💡 **Quando usar:** Quando você quer reutilizar os tipos dos parâmetros de uma função.

```
function greet(name: string, age: number) {  
  console.log(`Hello, ${name}! You are ${age} years old.`);  
}  
  
// [string, number]  
type GreetParams = Parameters<typeof greet>;
```

NonNullable<T>: Remove **null** e **undefined** de um tipo.

💡 **Quando usar:** Quando você quer garantir que um tipo não contenha **null** ou **undefined**.

```
type User = string | null | undefined;  
type ValidUser = NonNullable<User>; // string
```

Extract<T, U>: Extrai de um tipo todas as propriedades que são atribuíveis a outro tipo.

💡 **Quando usar:** Para criar subtipos específicos.

```
type Status = "active" | "inactive" | "suspended";  
  
// "active" | "suspended"  
type ActiveStatus = Extract<Status, "active" | "suspended">;
```

Resumo Visual (Tabela de Tipos Utilitários)

<code>Partial<T></code>	Torna todas as propriedades opcionais
<code>Required<T></code>	Torna todas as propriedades obrigatórias
<code>Readonly<T></code>	Torna todas as propriedades somente leitura
<code>Pick<T, K></code>	Seleciona propriedades específicas
<code>Omit<T, K></code>	Remove propriedades específicas
<code>Record<K, T></code>	Cria um objeto com chaves específicas
<code>ReturnType<T></code>	Tipo de retorno de uma função
<code>Parameters<T></code>	Tipos dos parâmetros de uma função
<code>NonNullable<T></code>	Remove null e undefined
<code>Extract<T, U></code>	Extraí tipos comuns entre dois tipos

Os tipos utilitários do TypeScript facilitam muito a vida dos desenvolvedores, tornando o código mais seguro, reutilizável e fácil de manter.

Entender e aplicar esses tipos no seu dia a dia ajuda a aproveitar ao máximo o TypeScript!

Para aprender sobre outros tipos utilitários, acesse [TypeScript: Documentation - Tipos Utilitários](#)

BONS ESTUDOS

